FreeRTOS 开发:
I 初识 FreeRTOS
FreeRTOS 简介:

轻量级实时操作系统

性:
实时操作系统
外设中断
任务执行顺序
最高优先级

为什么 ESP 前提要 FreeRTOS:
Esp-IDF 使用了 FreeRTOS.
gpio → 某个RTOS → IDF
RTOS → recv → 用户不懂底层 RTOS

RTOS 的特点
① 实时性: key_int.
盗 LX1 → virtual Timeline.
real
printf
② 任务调度: t1,2,3 → T.
t1 t2 t3 t1 ……
T T T T
③ 确定性:
④ 多任务管理:

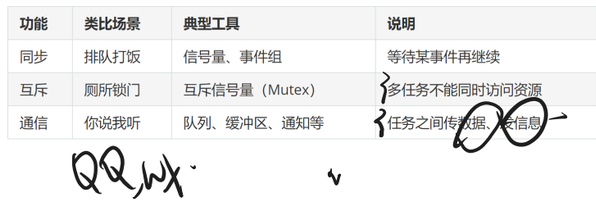
II. 任务的基本概念
单程 → application, 应用程序
task1 task2
FreeRTOS, task ⟷ thread
任务状态:
Running: 正在运行.
not running: 没有运行
ready:
blocked:
suspended: 废代码 → 整个流程
优先级: 同等优先级 → 任务的轮流 速度.
非等同 优先级. 执行
task1 与 task2
栈:
task → 函数,
task1 与 task2.
记忆功能

同步、互斥与通信:
① 排队打饭:
vTaskDelay( )
队头

食堂打饭要排队，一个人打完另一个人才开始。这就像一个任务做完，释放一个信号，另一个任务等着这个信号再开始。

· 信号量, 二值信号量, 计数信号量, 队尾
· 事件组.
互斥
互斥信号
① 关
② 关
FreeRTOS:
互斥信号量 (Mutex)
递归互斥锁 (Recursive Mutex)

t1 ⇄ t2
QQ_WX
交换数据



功能	类比场景	典型工具	说明
同步	排队打饭	信号量、事件组	等待某事件再继续
互斥	厕所锁门	互斥信号量（Mutex）	多任务不能同时访问资源
通信	你说我听	队列、缓冲区、通知等	任务之间传数据、发信息

- **消息队列（Queue）**
- **流缓冲区（Stream Buffer）**
- **消息缓冲区（Message Buffer）**
- **任务通知（Task Notification）**

QQ_WX

② 任务的创建与删除
✓ 任务: 函数.
int Count;
static int Count;

```
void ATaskFunction(void
*pvParameters)
```

pvParameters
task1 → ATask…
task2 → ATask…
局部变量 栈
A a B b

任务函数

```
BaseType_t xTaskCreate( TaskFunction_t pxTaskCode,
                        const char * const pcName, /*lint !e971 Unqualified
char types are allowed for strings and single characters only. */
                        const configSTACK_DEPTH_TYPE usStackDepth,
                        void * const pvParameters,
                        UBaseType_t uxPriority,
                        TaskHandle_t * const pxCreatedTask )
```

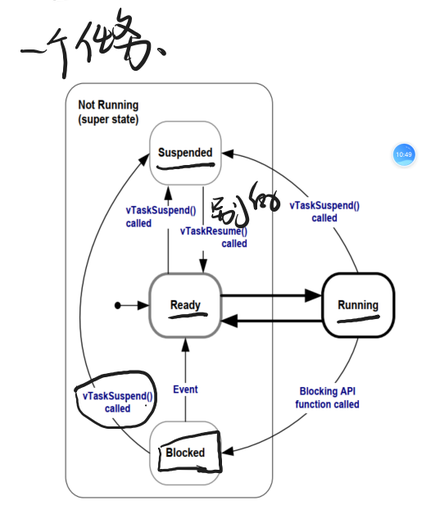
任务名称 str.
栈的大小、
32位
1 → 1word → 4字节
1000 → 4000字节
0 ~ configMAX_PRIOPITIEY-1
越小优先级越大.

③ Tick:
优先级.
同 2 → 轮流
人有心跳, → 基本稳定.
定时器 → 间隔 → 判断 → Tick.
10ms 时钟中断
configTICK_RATE_HZ.
configTICK_RATE_HZ
100 → 10ms.
t1 t2 t3
时间片
内核
task2
task1
tick.
t1 t2 t3
tick 中断函数
选择要执行的任务.

延时函数:
① Tick.
pdMS_TO_TICKS( ) ms.
② vTaskDelay(TICK)
MS / portTICK_PERIOD_MS
② vTaskDelayUntil()
周期性地运行
读: 20ms 30ms
task: 80ms 70ms
100ms
tick
vTaskDelayUntil(pxPreviousWakeTime, xTimeIncrement)
时间基准. 周期
什么时候开始第一次运行
TickType_t xTaskGetTickCount(void)
获取当前 Tick

任务状态:
运行
非运行 阻塞. 就绪 挂起
阻塞:
文件
信息
1 2
时间:
同步事件.
睡觉 3、4, 火灾, 暂停.
vTaskSuspend()
void vTaskSuspend(TaskHandle_t xTaskToSuspend)
暂停自己 NULL.
退出 → 另队.
T1 T2
一个任务.

void vTaskResume(TaskHandle_t xTaskToResume)
BaseType_t xTaskResumeFromISR(TaskHandle_t xTaskToResume)
ISR → T


Not Running (super state)
Suspended
vTaskSuspend() called
vTaskResume() called
vTaskSuspend() called
Ready
Running
Event
Blocking API function called
vTaskSuspend() called
Blocked
到的

任务调度器:
调度器就是一个任务的大管家
挑选优先级最高的任务，
并且这个任务还必须处在Ready
使用时间片轮转调度 configUSE_TIME_SLICING
- 任务创建/删除
- 任务被唤醒（例如延时结束、获取资源成功）
- 当前任务调用了 `taskYIELD()` 主动放弃
- 被抢占（高优先级任务准备就绪）
- 系统 tick（如果启用了抢占式调度）
Ready
抢占式调度: 高优先级任务 → 低
configUSE_PREEMPTION
协作式调度:
configUSE_PREEMPTION
高 低
系统 Tick 定时器
SysTick 定时器中断
vTaskStartScheduler()
开启调度器

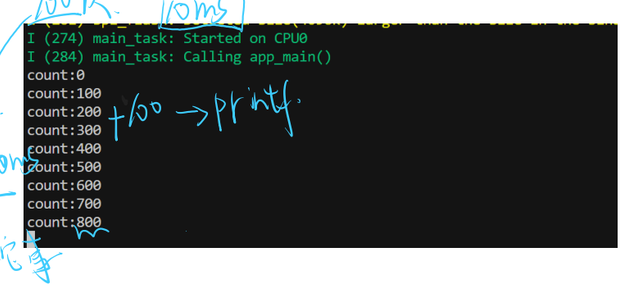
空闲任务与钩子函数:
① 空闲任务:
CPU 无任务去干 → 空闲任务
闲下来 崩掉.
C: int main( )
{ while(1)
{ }
return 0;
}
ESP
CPU: 每核都有一个空闲任务
钩子定义 malloc 申请内存 → 失败.
空闲钩子函数
传统 FreeRTOS( )
IDF ESP
钩子函数. 指定哪个 CPU 核心 数字

```
esp_err_t esp_register_freertos_idle_hook_for_cpu(esp_freertos_idle_cb_t hook, BaseType_t cpu_id);
esp_err_t esp_deregister_freertos_idle_hook_for_cpu(esp_freertos_idle_cb_t hook, BaseType_t cpu_id);
```

Idle 任务 busy → task.

```
typedef bool (*esp_freertos_idle_cb_t)(void);
```

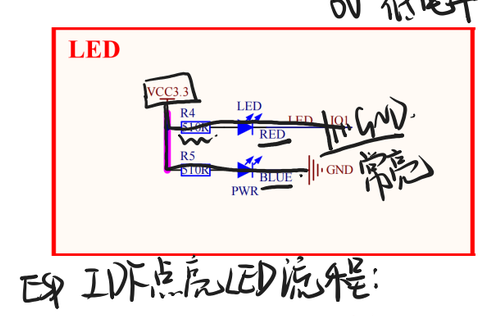
Idle: Count++; 1000ms
100次 10ms
main:
printf → 1s.
vdelay 1000ms
无任何其它事



```
I (274) main_task: Started on CPU0
I (284) main_task: Calling app_main()
count:0
count:100
count:200
count:300
count:400
count:500
count:600
count:700
count:800
```

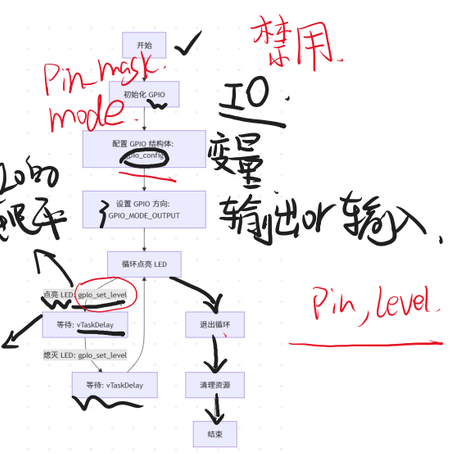
+100 → printf.

I. IDF 点亮第1个LED灯
GND IO1 3.3V
进行高电平
LED → 点灯.
0V 低电平


LED
VCC3.3
R4
510R
LED
RED
LED
IO1
GND
R5
510R
BLUE
PWR
GND
常亮

飞线
ESP IO.

ESP IDF 点亮 LED 流程:


开始
初始化 GPIO
配置 GPIO 结构体: gpio_config
设置 GPIO 方向: GPIO_MODE_OUTPUT
循环点亮 LED
点亮 LED: gpio_set_level
等待: vTaskDelay
熄灭 LED: gpio_set_level
等待: vTaskDelay
退出循环
清理资源
结束

禁用.
Pin_mask.
mode.
IO.
变量
输出 or 输入.
设置 IO 的 高低电平
FreeRTOS
Pin, level.

II. 将驱动代码与主代码(main.c)分离
原项目结构

```
e:\esp examples\examples\peripherals\gpio\generic_gpio\
├— main/          # 主程序目录
│  ├— CMakeLists.txt # 主程序构建配置
│  └— main.c        # 主程序代码
└— CMakeLists.txt     # 项目根构建配置
```

led 驱 main.c

更改项目结构

```
e:\esp examples\examples\peripherals\gpio\generic_gpio\
├— main/          # 主程序目录
│  ├— CMakeLists.txt # 主程序构建配置
│  └— main.c        # 主程序代码
├— components/       # 组件目录
│  └— my_component/  # 自定义组件
│     ├— include/   # 组件头文件
│     ├— src/       # 组件源文件
│     └— CMakeLists.txt # 组件构建配置
└— CMakeLists.txt     # 项目根构建配置
```

LED_driver.
src, include.

III. 读取 GPIO 使 Key.